

Aula 02

# Introdução ao CSS

Front end

---

Prof. Jezer Machado de Oliveira

Material original: Prof. Augusto César Oliveira (Universidade de Pernambuco)

# O objetivo da aula de hoje...

- Compreender o que é o CSS e seu papel no desenvolvimento web;
- Identificar a estrutura de uma regra CSS (seletor, propriedades e valores);
- Aplicar estilos básicos a elementos HTML usando seletores de tipo, classe e ID;
- Controlar o tamanho e o posicionamento de elementos HTML usando as propriedades de box model;

# O objetivo da aula de hoje...

- Compreender o modelo de layout Flexbox;
- Utilizar propriedades de fonte para estilizar textos;
- Aplicar cores a elementos usando as propriedades color e background-color;
- Identificar as pseudoclasses comuns, como :hover, :active, :visited e :focus; e
- Criar layouts responsivos que se adaptem a dispositivos diversos.

1.

# Conceitos básicos

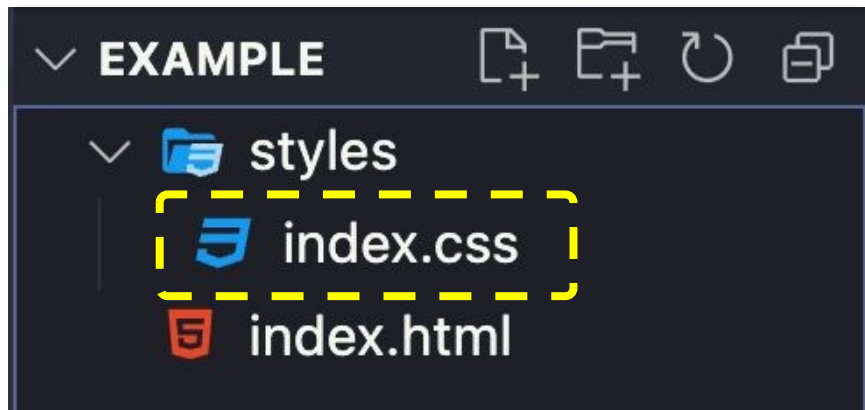
Introdução ao CSS

# O que é CSS?

- O **CSS** (*Cascading Style Sheets*) é uma **linguagem de estilo** utilizada para definir a **apresentação e o design de páginas web**.
- É a ferramenta essencial para **controlar o layout**, as **cores**, as **fontes** e outros aspectos visuais de um **site**.

## Por que usar CSS?

- Separar a **estrutura (HTML)** do **estilo (CSS)** permite maior **flexibilidade** e **manutenção do código**.

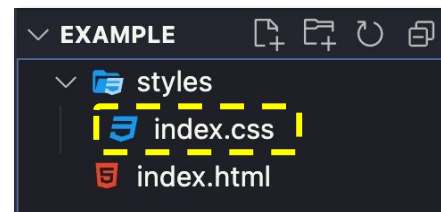


# Estilos externos

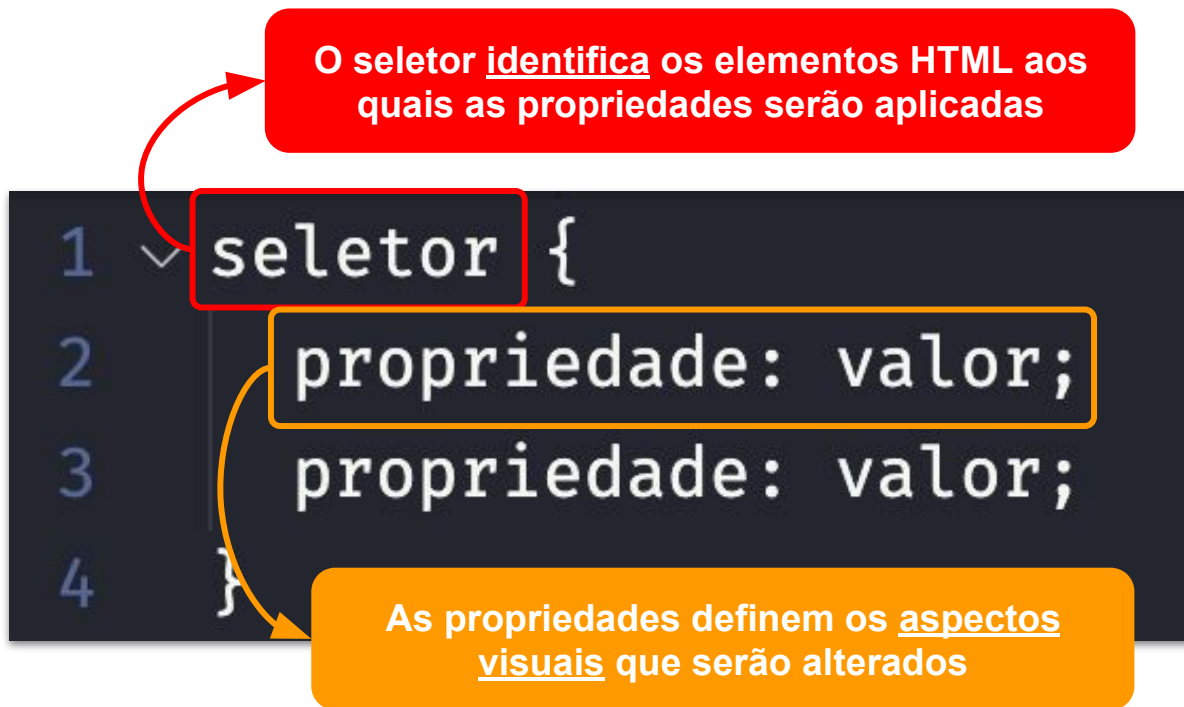
```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <link rel="stylesheet" type="text/css" href="./styles/index.css">
8   <title>Document</title>
9 </head>
10
11 <body>
12   <!-- Conteúdo do corpo -->
13 </body>
14
15 </html>
```

A tag `<link>` permite adicionar estilos definidos em um arquivo CSS separado

Caminho do arquivo CSS



# Sintaxe básica do CSS





2.

# Seletores CSS

Introdução ao CSS

## Seletores de **tipo**

- Aplicam estilos a **elementos** com base em seu **tipo HTML**.

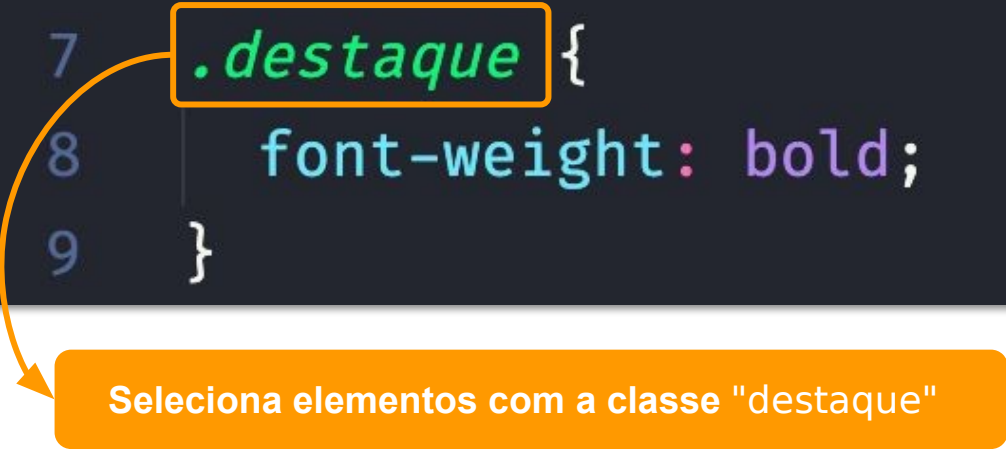
```
1  /* Selecciona todos os elementos <p> */  
2  p {  
3      color: blue;  
4  }
```

Selecciona todos os elementos <p>

## Seletores de **classe**

- Aplicam estilos a **elementos** com base em uma **classe atribuída**.

```
6  /* Selecciona elementos com a classe "destaque"  
7  .destaque {  
8      font-weight: bold;  
9  }
```



Selecciona elementos com a classe "destaque"

## Seletores de **ID**

- Aplicam estilos a **um elemento específico** com base em seu **ID exclusivo**.

```
13  #cabecalho {  
14      font-size: 24px;  
15  }
```

Seleciona o elemento com o ID "cabecalho"

## Exemplo de propriedades

Define a cor do texto como  
vermelho

Define do tamanho da fonte  
como 18 pixels

Define uma margem de 10 pixels  
para todos os lados do elemento

```
1  √ p {  
2    color: red;  
3  }  
4  
5  √ h1 {  
6    font-size: 18px;  
7  }  
8  
9  √ div {  
10   margin: 10px;  
11 }
```

3.

# Box Model

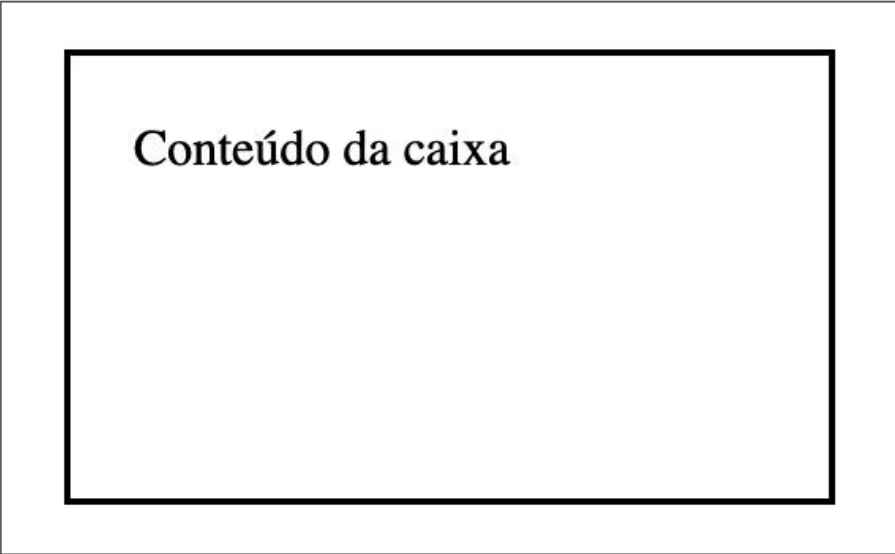
Introdução ao CSS

# O que é Box Model?

- O **Box Model** é um conceito fundamental no CSS que descreve **como os elementos HTML são renderizados e ocupam espaço na página.**
- O modelo é composto por quatro partes principais: **conteúdo**, **padding**, **border** e **margin**.

# Explicação do Box Model

```
1  .caixa {  
2    width: 200px;  
3    height: 100px;  
4    padding: 20px;  
5    border: 2px solid black;  
6    margin: 20px;  
7  }
```

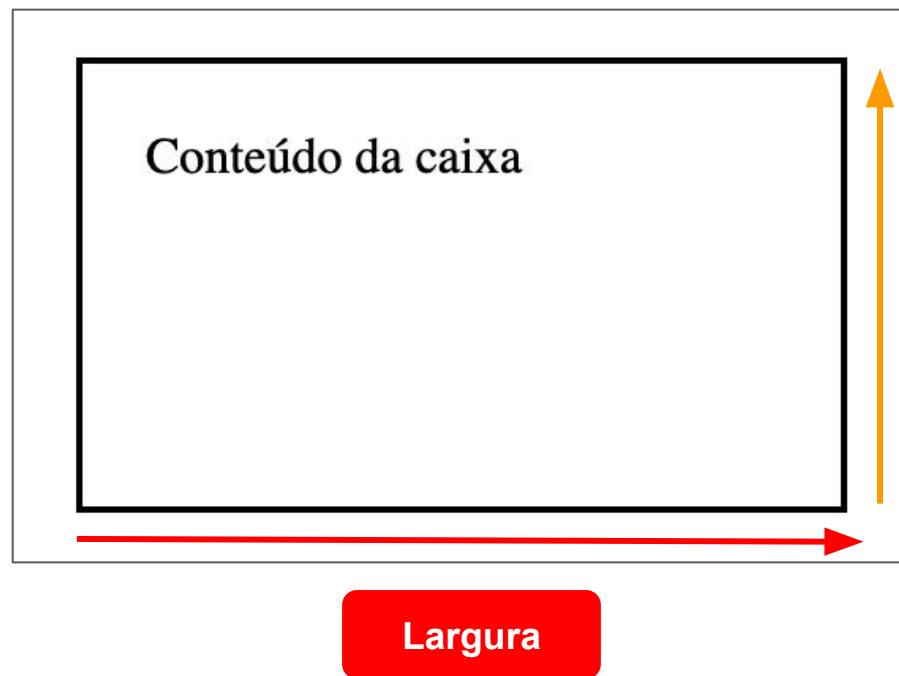


Conteúdo da caixa



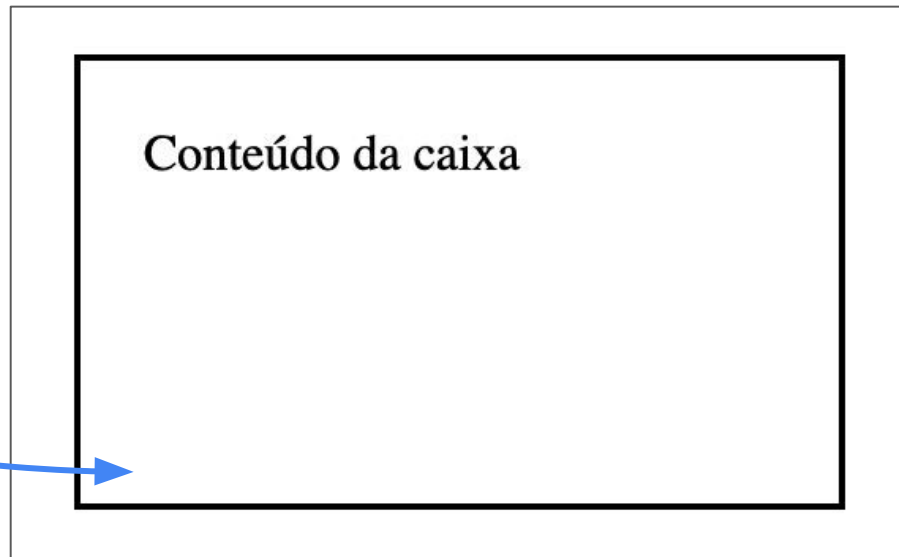
## Altura e largura da caixa

```
1  .caixa {  
2    width: 200px;  
3    height: 100px;  
4    padding: 20px;  
5    border: 2px solid black;  
6    margin: 20px;  
7  }
```



## Preenchimento da caixa

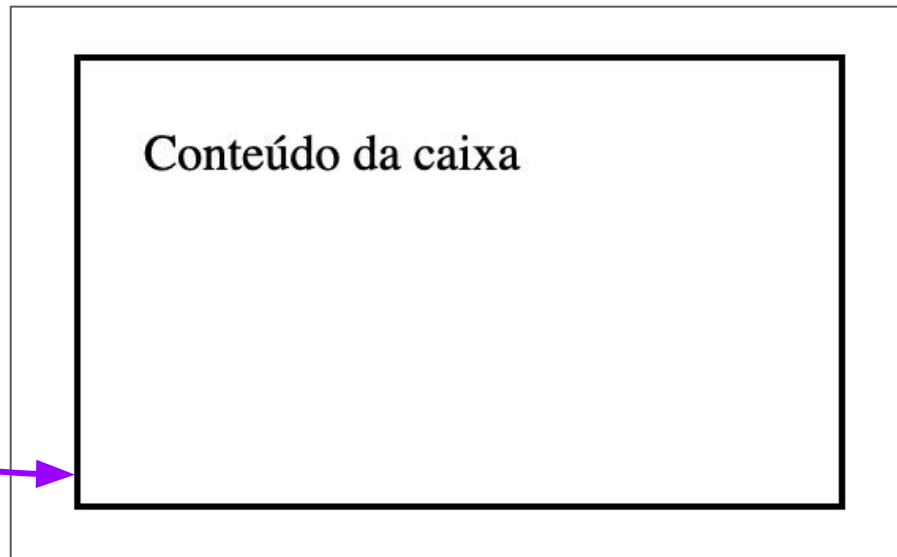
```
1  .caixa {  
2    width: 200px;  
3    height: 100px;  
4    padding: 20px;  
5    border: 2px solid black;  
6    margin: 20px;  
7  }
```



Preenchimento (padding): é uma área transparente ao redor do conteúdo que separa o conteúdo da borda

## Borda da caixa

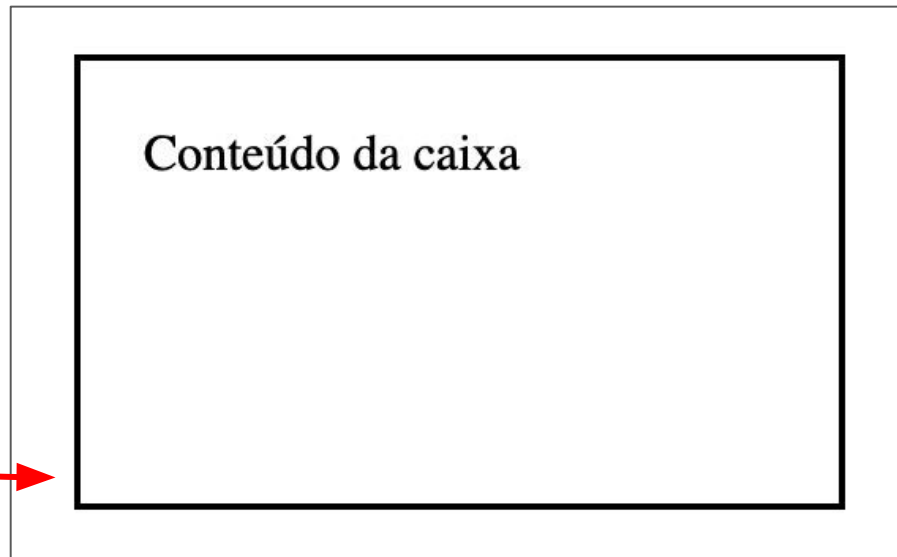
```
1  .caixa {  
2    width: 200px;  
3    height: 100px;  
4    padding: 20px;  
5    border: 2px solid black;  
6    margin: 20px;  
7  }
```



**Borda (border):** é uma linha que circunda o conteúdo e o preenchimento

## Margem da caixa

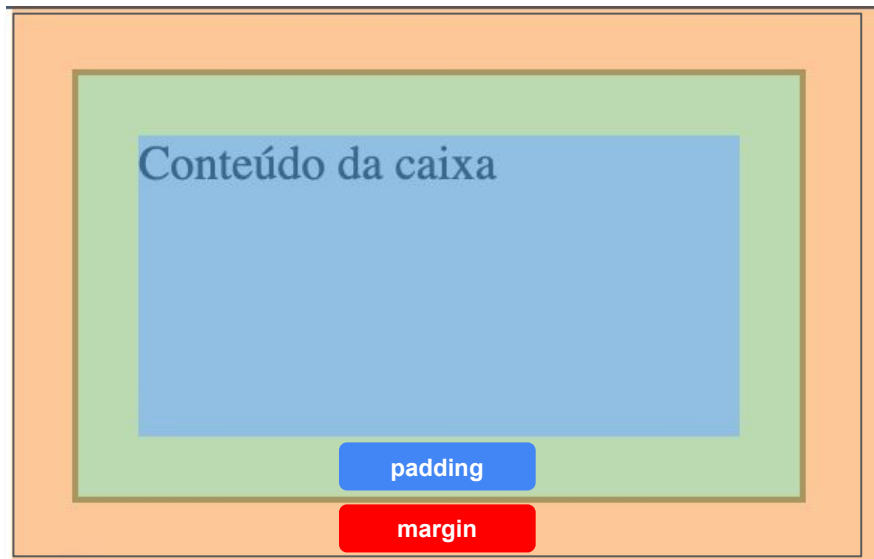
```
1  .caixa {  
2    width: 200px;  
3    height: 100px;  
4    padding: 20px;  
5    border: 2px solid black;  
6    margin: 20px;  
7  }
```



**Margem (margin): é uma área transparente fora da borda que separa o elemento de outros elementos na página**

## Visão geral do Box Model

```
1  .caixa {  
2    width: 200px;  
3    height: 100px;  
4    padding: 20px;  
5    border: 2px solid black;  
6    margin: 20px;  
7  }
```



4.

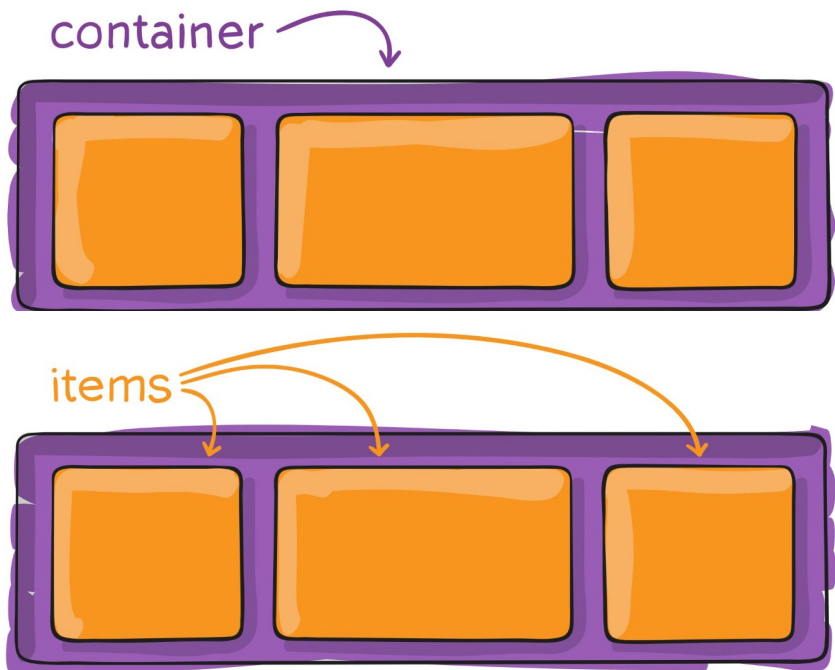
# Flexbox

Introdução ao CSS

# O que é Flexbox?

- **Flexbox** é um recurso do CSS que permite **criar layouts flexíveis** e **responsivos** em páginas da web.
- Possibilita **controlar o tamanho**, a **ordem**, o **alinhamento** e o **espaçamento dos elementos HTML** de maneira flexível.

# Propriedades do Flexbox

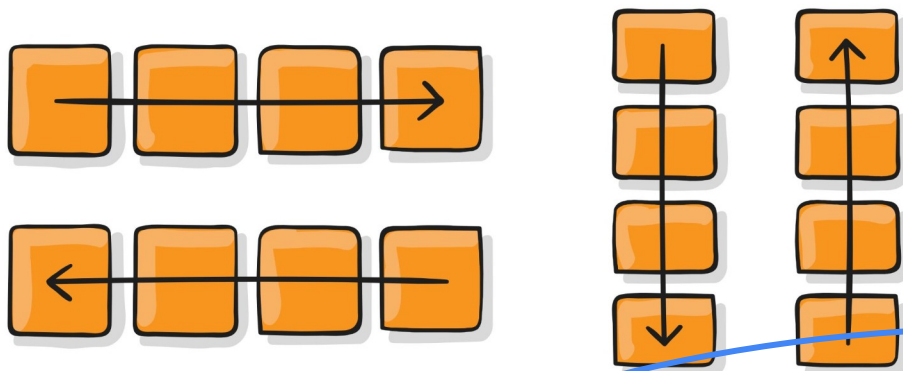


```
1  .container {  
2      display: flex;  
3  }
```

**display: flex** define um contexto flexível para todos os seus filhos diretos



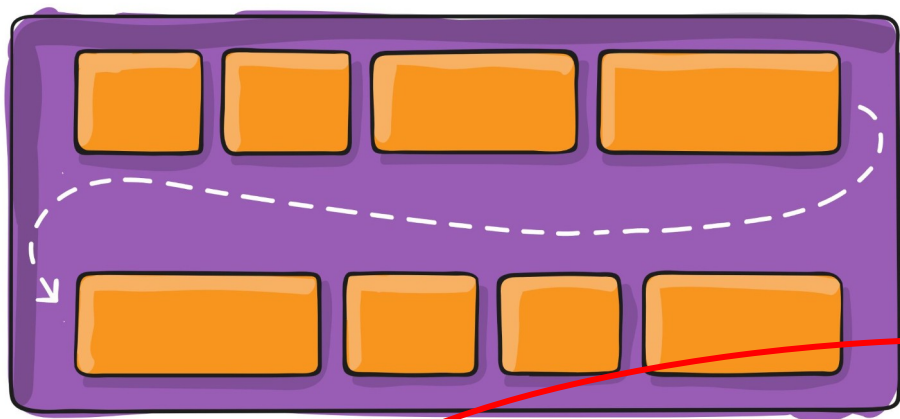
## flex-direction



flex-direction define o eixo principal, definindo a direção dos itens

```
1 .container {  
2   flex-direction: row | row-reverse | column | column-reverse;  
3 }
```

# flex-wrap



**flex-wrap** por padrão, todos os itens tentarão caber em uma linha. Isso pode ser mudado

```
1  .container {  
2    flex-wrap: nowrap | wrap | wrap-reverse;  
3  }
```

# justify-content

```
1 .container {  
2   justify-content: flex-start | flex-end | center |  
3     space-between | space-around | space-evenly;  
4 }
```

justify-content define o alinhamento dos itens ao longo do eixo principal

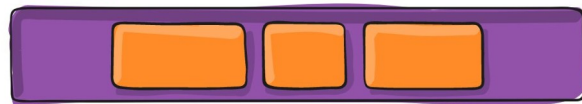
flex-start



flex-end



center



space-between



space-around



space-evenly

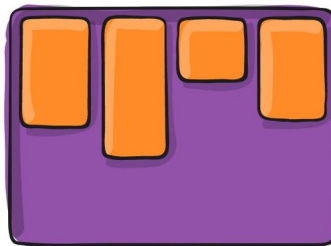


# align-items

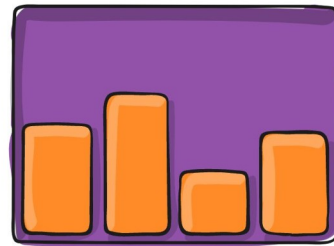
```
1 .container {  
2   align-items: stretch | flex-start  
3   | flex-end | center | baseline;  
4 }
```

align-items define o  
alinhamento vertical dos itens  
dentro do contêiner

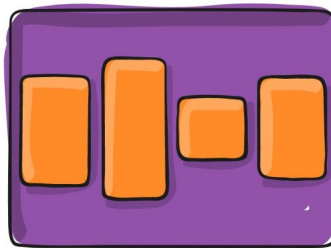
flex-start



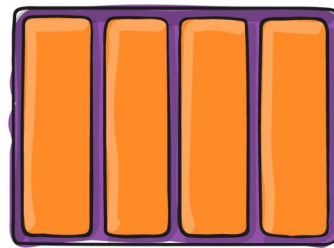
flex-end



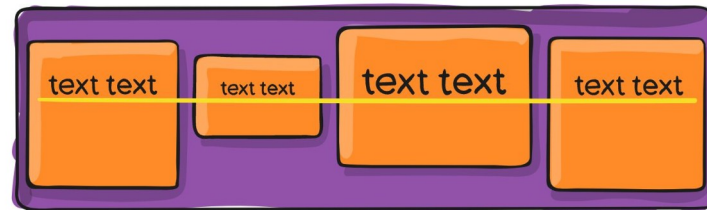
center



stretch



baseline

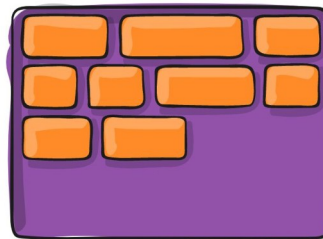


# align-content

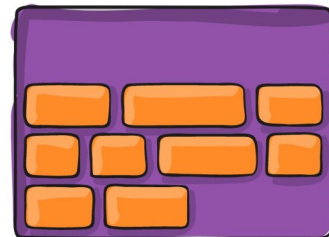
```
1 .container {  
2   align-content: flex-start | flex-end | center |  
3     space-between | space-around | stretch;  
4 }
```

align-content define o alinhamento dos itens quando há espaço extra no eixo cruzado

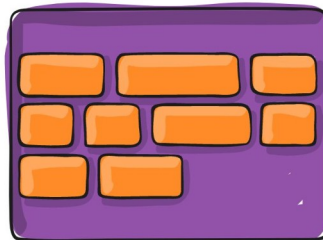
flex-start



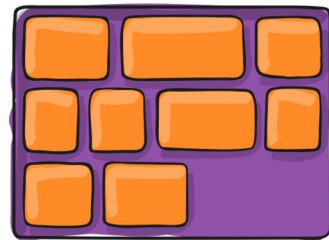
flex-end



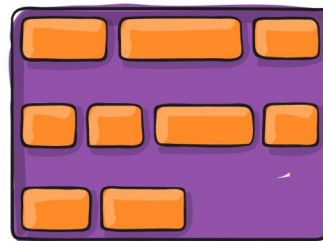
center



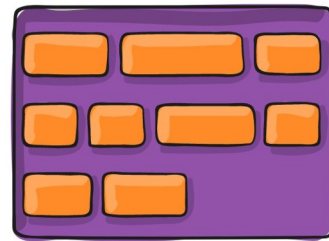
stretch



space-between

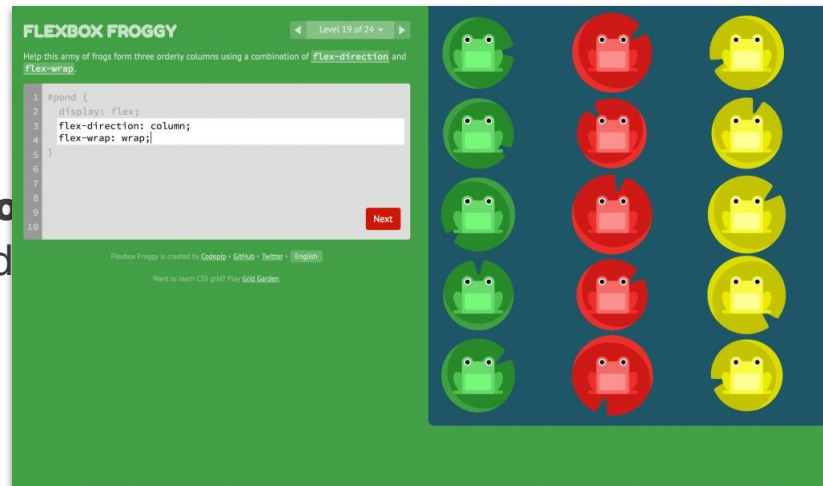


space-around



# Prática de Flexbox com Flexbox Froggy

- <https://flexboxfroggy.com/>.
- **24 fases** com diferentes desafios.
- Facilita a **compreensão** dos **fundamentos** e das **propriedades** do Flexbox através da **prática**.



# 5. Tipografia

Introdução ao CSS

# Propriedades de fonte

color define a cor da fonte

font-family define a família de fontes a ser usada

font-size define o tamanho da fonte

```
1 p {  
2   color: red;  
3   font-family: Arial, sans-serif;  
4   font-size: 24px;  
5 }
```

Lorem ipsum dolor sit amet.



## Outras propriedades de fonte

**font-weight** define a  
espessura da fonte

**font-style** define o estilo da  
fonte

**text-decoration** define a  
decoração do texto

**text-transform** define a  
transformação do texto

```
1  p {  
2      /* Outras propriedades */  
3      font-weight: bold;  
4      font-style: italic;  
5      text-decoration: underline;  
6      text-transform: uppercase;  
7  }
```

**LOREM IPSUM DOLOR SIT AMET.**

6.

# Cores e fundos

Introdução ao CSS

# Propriedades de cor

Utilização de cores por nome

Utilização de cores por hexadecimal

Utilização de cores por rgb

```
1  .container {  
2    width: 200px;  
3    height: 100px;  
4  
5    background-color: red;  
6    background-color: #FF0000;  
7    background-color: rgb(255, 0, 0);  
8  }
```



# Opacidade

opacity define a opacidade de um elemento

```
1  .container {  
2      width: 200px;  
3      height: 100px;  
4  
5      background-color: red;  
6      opacity: 0.5;  
7  }
```



7.

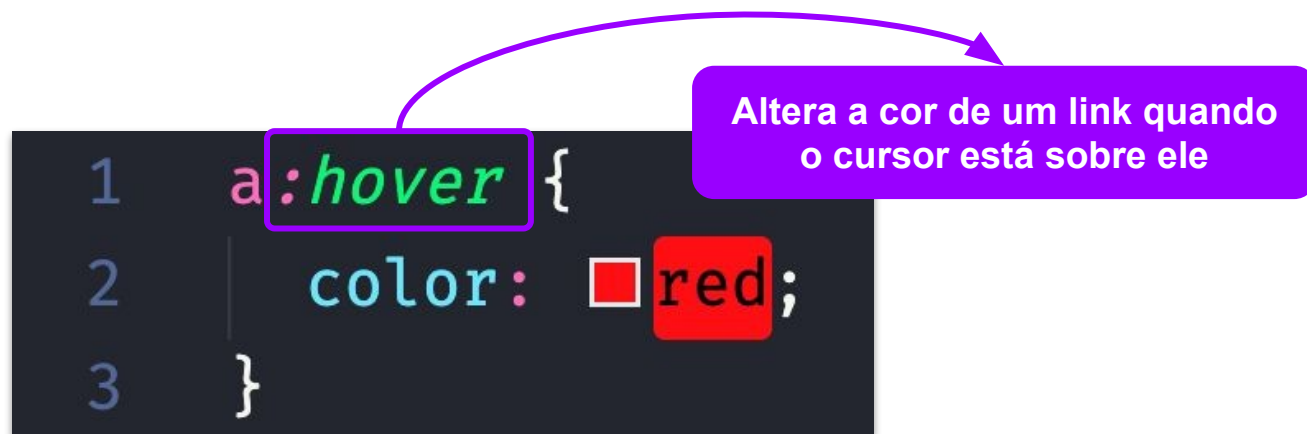
# Pseudoclasses

Introdução ao CSS

# Pseudoclasses

- As pseudoclasses permitem **selecionar elementos** com base em **estados** ou **interações** específicas.
- Exemplos:
  1. **:hover:** seleciona um elemento quando o cursor está sobre ele.
  2. **:active:** seleciona um elemento quando ele é clicado.
  3. **:visited:** seleciona um link já visitado.
  4. **:focus:** seleciona um elemento quando ele recebe foco.

## Exemplo prático de uso de pseudoclasses



[Ir para Seção 1](#)

[Ir para Seção 1](#)

8.

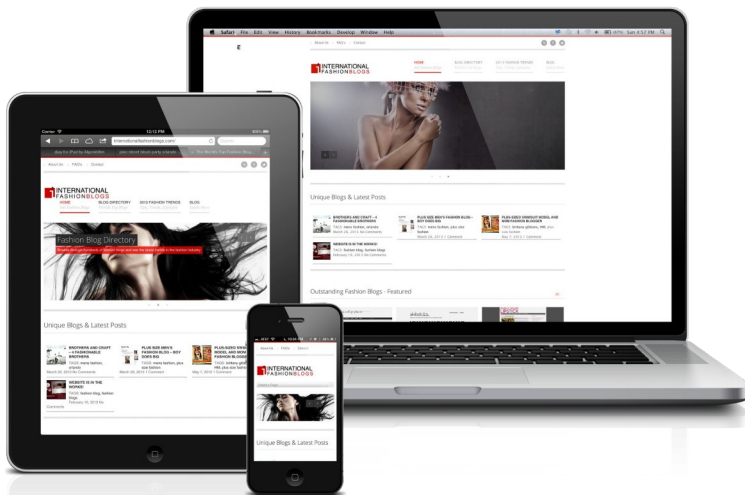
# Responsividade e media queries

Introdução ao CSS



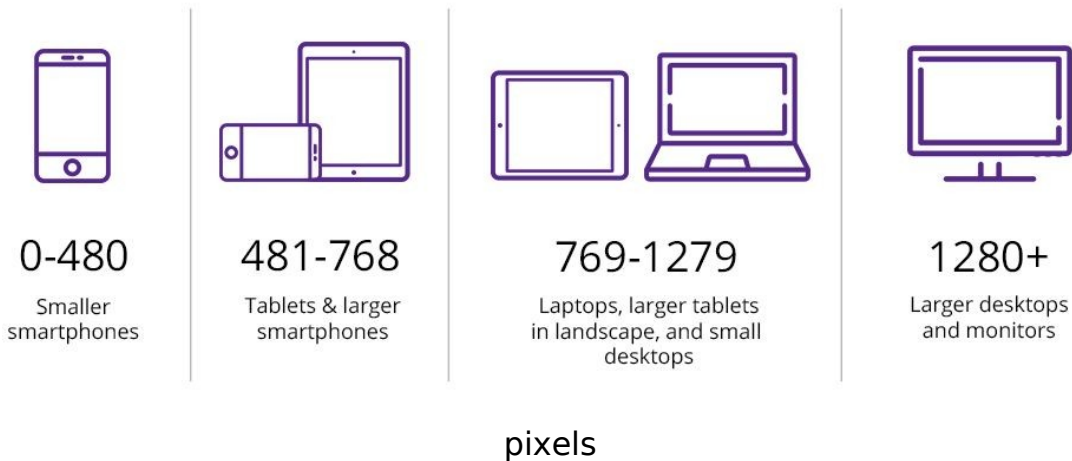
# O que é responsividade?

- A responsividade é a **capacidade de um site se adaptar a diferentes dispositivos e tamanhos de tela.**



# O que são media queries?

- As **media queries** são uma **técnica do CSS** que permite **aplicar diferentes estilos** Com base em pontos de interrupções (**breakpoints**) e nas **características do dispositivo**.



## Exemplo de media query para diferentes tamanhos

Estilos padrão para todas as telas

Estilos para telas menores que 768px

Estilos para telas menores que 480px

```
1  h1 {  
2    font-size: 24px;  
3  }  
4  
5  @media (max-width: 768px) {  
6    h1 {  
7      font-size: 20px;  
8    }  
9  }  
10  
11 @media (max-width: 480px) {  
12   h1 {  
13     font-size: 16px;  
14   }  
15 }
```

## Pontos de interrupção comuns

- **320px**: dispositivos móveis pequenos.
- **768px**: tablets e dispositivos de tamanho médio.
- **1024px**: laptops e monitores de desktop.

9.

# Considerações finais

Introdução ao CSS

## O que aprendemos hoje?

- O que é o CSS e seu papel no desenvolvimento web;
- Identificar a estrutura de uma regra CSS (seletor, propriedades e valores);
- Aplicar estilos básicos a elementos HTML usando seletores de tipo, classe e ID;
- Controlar o tamanho e o posicionamento de elementos HTML usando as propriedades de box model;

## O que aprendemos hoje?

- O modelo de layout Flexbox;
- Utilizar propriedades de fonte para estilizar textos;
- Aplicar cores a elementos usando as propriedades color e background-color;
- Identificar as pseudoclasses comuns, como :hover, :active, :visited e :focus; e
- Criar layouts responsivos que se adaptem a dispositivos diversos.